Linear Algebra: Computational Labs

UATX Spring 2025

12 labs — instructor selects 10 for an 11-week course

Overview. Each lab is a self-contained Jupyter notebook intended for a two-hour session. Labs are ordered to follow the lecture notes closely. Labs with an existing notebook are marked with their filename; the remaining labs are to be built. The table below summarises the full sequence.

#	Title	Week	Notebook	Status
1	Vector Spaces in Python	2	`lab01_vector_spaces.ipynb`	to build
2	Linear Maps and Their Matrices	3	`lab02_linear_maps.ipynb`	to build
3	Image, Kernel, and Gaussian Elimination	4–5	`lab03_im_ker_gaussian.ipynb`	exists
4	Solving $Ax = b$: the Four Cases	5	`lab04_solving_axb.ipynb`	to build
5	Polynomial Regression	5–6	`lab05_polynomial_regression.ipynb`	exists
6	Inner Products, Norms, Gram–Schmidt	7	`lab06_gram_schmidt.ipynb`	to build
7	Four Fundamental Subspaces	7–8	`lab07_four_subspaces.ipynb`	to build
8	Change of Basis	8	`lab08_change_of_basis.ipynb`	to build
9	Eigenvalues via Rayleigh Quotients	9	`lab09_rayleigh_quotients.ipynb`	exists
10	SVD: Geometry and Compression	10	`lab10_svd.ipynb`	exists
11	Image Processing as Linear Algebra	10–11	`lab11_edge_detection.ipynb`	exists
12	Matrix Denoising via SVD	11	`lab12_denoising.ipynb`	to build

Suggested schedule for a 10-lab course. Drop one of Labs 1, 2, or 8 if time is short. Lab 12 can replace Lab 11 for a more theoretical emphasis on spectral methods.

Lab 1. Vector Spaces in Python

Suggested week: Week 2 **Notebook:** `lab01_vector_spaces.ipynb` **Prerequisites:** Notes §1.1–§1.2 (pages 1–6)

Background. A vector space over a field F is a set V with operations of addition and scalar multiplication satisfying eight axioms. The simplest examples are F^n, the space of polynomials P_n, and spaces of functions. A subset $W \subseteq V$ is a subspace if and only if it is closed under addition and scalar multiplication (the "one-condition test").

Learning objectives.

- Translate the abstract vector space axioms into concrete Python checks.
- Recognise which candidate sets are subspaces and which are not.
- Visualise linear combinations and span in $\mathbb{R}^2$ and $\mathbb{R}^3$.
- Build intuition for why isomorphic spaces ($P_n \cong \mathbb{R}^{n+1}$) behave identically.

Tasks.

1. Implement `check_vector_space_axioms(V, add, scale, F, samples=50)` that draws random elements and verifies all eight axioms stochastically. Run it on $\mathbb{R}^3$, P_2, and the set of 2×2 matrices.
2. For each of the following subsets of $\mathbb{R}^3$, use the one-condition test to determine (analytically and verify numerically) whether it is a subspace: (a) $\{x : x_1 + x_2 = 0\}$, (b) $\{x : x_1 x_2 = 0\}$, (c) $\{x : \|x\| = 1\}$, (d) $\{x : Ax = 0\}$ for a fixed matrix A.
3. Given $v_1, \ldots, v_k \in \mathbb{R}^n$, write code that generates m random linear combinations and plots them. Show that $\operatorname{span}\{(1,0),(0,1)\} = \mathbb{R}^2$ but $\operatorname{span}\{(1,1)\}$ is only a line.
4. Represent polynomials $p \in P_2$ as coefficient vectors in $\mathbb{R}^3$. Evaluate the isomorphism: pick two random polynomials, add them in both representations, and confirm the results match.

Extensions.

- Implement the *direct sum* test: given subspaces $U, W \subseteq \mathbb{R}^n$ expressed as column-span matrices, check whether $U \cap W = \{0\}$.
- Explore $\mathbb{Z}_2^n$ as a vector space over $\mathbb{Z}_2$; verify the axioms.

Lab 2. Linear Maps and Their Matrices

Suggested week: Week 3 **Notebook:** `lab02_linear_maps.ipynb` **Prerequisites:** Notes §1.5 (pages 11–14)

Background. Every linear map $T : V \to W$ between finite-dimensional spaces is completely determined by its action on a basis: if $\{e_1, \ldots, e_n\}$ is a basis of V then the *matrix of* T is $M_T = [T(e_1) \mid \cdots \mid T(e_n)]$. Composition of linear maps corresponds to matrix multiplication. A canonical example is the differentiation operator $D : P_n \to P_{n-1}$ and the integration operator $J : P_{n-1} \to P_n$; one can show $D \circ J = \mathrm{id}$.

Learning objectives.

- Build the matrix of a linear map from its action on a basis.
- Verify that composition of maps corresponds to matrix multiplication.
- Explore differentiation and integration as explicit matrices on P_n.
- Understand that T is determined by finitely many values.

Tasks.

1. Represent P_n as $\mathbb{R}^{n+1}$ (coefficient vectors). Write `diff_matrix(n)` that returns the $(n \times (n+1))$ matrix of $D : P_n \to P_{n-1}$ in the monomial basis. Verify: applying the matrix to $[0, 0, 1, 0]^T$ (representing x^2) gives $[0, 2, 0]^T$ (representing $2x$).
2. Similarly build `int_matrix(n)` for $J(p)(x) = \int_0^1 x\, p(xt)\, dt$. Compute $D \cdot J$ and verify it equals the identity.
3. Write `matrix_of_T(T, basis)` that constructs M_T for any callable linear map T and a list of basis vectors. Apply it to (a) reflection across the line $y = x$ in $\mathbb{R}^2$, (b) rotation by $\theta = \pi/4$, (c) projection onto a given vector v.
4. Compose two maps $T_1, T_2 : \mathbb{R}^3 \to \mathbb{R}^3$ defined by your matrices and verify $M_{T_2 \circ T_1} = M_{T_2} \cdot M_{T_1}$.

Extensions.

- Compute D^k (the k-th derivative operator) using matrix powers and verify against symbolic differentiation with `sympy`.
- Show that the space $L(V, W)$ of linear maps is itself a vector space by defining addition and scalar multiplication of maps and checking axioms.

Lab 3. Image, Kernel, and Gaussian Elimination

Suggested week: Weeks 4–5 **Notebook:** `lab03_im_ker_gaussian.ipynb` **Prerequisites:** Notes §1.6 (pages 15–21)

Background. For a matrix $A \in F^{m \times n}$, the *image* $\mathrm{im}(A) = \{Ax : x \in F^n\}$ and *kernel* $\ker(A) = \{x : Ax = 0\}$ are both subspaces. Column reduction on $[A \mid I_n]$ simultaneously yields a basis for $\mathrm{im}(A)$ (the pivot columns) and a basis for $\ker(A)$ (the kernel part of the non-pivot columns).

A striking numerical example is the *Hilbert matrix* $H_{ij} = 1/(i + j - 1)$: it is theoretically invertible for all n, but floating-point column reduction declares H_{20} to have rank 12 because its condition number exceeds 10^{17}.

Learning objectives.

- Implement column reduction on $[A \mid I_n]$ and read off bases for $\mathrm{im}(A)$ and $\ker(A)$.
- Solve $Ax = b$ using the extended algorithm $[A \mid I_n \mid -b]$.
- Understand the gap between exact (symbolic) and floating-point rank.
- Connect the algorithm to classical Gauss–Jordan elimination.

Tasks.

1. **(Already in notebook)** Run `column_reduce_image_kernel` on a rank-2 matrix and an invertible matrix. Identify pivot columns, free columns, image basis, and kernel basis.
2. **(Already in notebook)** Run `gauss_solve` on: (a) an invertible 3×3 system, (b) a consistent underdetermined system, (c) an inconsistent system. In each case identify the solution, the solution set, and the obstruction.
3. **(Already in notebook)** Compute the 20×20 Hilbert matrix exactly (using `sympy`) and verify $HH^{-1} = I$. Then pass H as a `float64` array to `column_reduce_image_kernel` and observe that it reports rank 12. Explain the discrepancy.
4. **(New)** Plot the numerical rank of H_n (as reported by both `np.linalg.matrix_rank` and your algorithm) for $n = 2, 4, \ldots, 20$. At what size does floating-point rank diverge from the true rank?

Extensions.

- Repeat the rank experiment with different tolerance thresholds `tol` in `column_reduce_image_kernel`. How sensitive is the result?
- Look up the closed-form formula for $\kappa(H_n)$ (condition number of the Hilbert matrix) and overlay the theoretical curve on your rank plot.

Lab 4. Solving $Ax = b$: The Four Cases

Suggested week: Week 5 **Notebook:** `lab04_solving_axb.ipynb` **Prerequisites:** Notes pages 20–21 (2×2 table)

Background. The solvability of $Ax = b$ and the structure of the solution set are governed by two independent binary questions:

1. Is $b \in \mathrm{im}(A)$? (existence)
2. Is $\ker(A)$ trivial? (uniqueness)

This gives a 2×2 table: unique solution, infinitely many solutions (coset $x_0 + \ker A$), no solution, or no solution. Importantly, the *least-squares* solution $\hat{x} = (A^\top A)^{-1} A^\top b$ always exists when A has full column rank and minimises $\|Ax - b\|$.

Learning objectives.

- Construct matrices and right-hand sides that realise each of the four cases.
- Parameterise the full solution set $x_0 + \ker(A)$ from the reduced form.
- Compute and interpret the least-norm solution when infinitely many solutions exist.
- Visualise solution sets in $\mathbb{R}^2$ and $\mathbb{R}^3$.

Tasks.

1. For each case in the 2×2 table, construct an explicit 3×3 (or 3×4) matrix A and vector b that realises it. Use `gauss_solve` from Lab 3 to confirm the classification.
2. For a 3×5 matrix with rank 2 and $b \in \mathrm{im}(A)$, extract one particular solution x_0 and a

basis $\{k_1, \ldots, k_3\}$ for $\ker(A)$. Verify that $x_0 + \alpha k_1 + \beta k_2 + \gamma k_3$ is a solution for random α, β, γ.

3. Implement `least_norm_solution(A, b)` using `np.linalg.lstsq` and verify it returns the solution x^* orthogonal to $\ker(A)$ (i.e. $x^* \perp k_i$ for all kernel basis vectors).
4. Plot the solution set of $x_1 + 2x_2 = 3$ in $\mathbb{R}^2$ as a line, and mark the least-norm solution as the point closest to the origin. Generalise to a plane in $\mathbb{R}^3$.

Extensions.

- Show that for any $b \in \mathrm{im}(A)$, the least-norm solution equals $A^\top(AA^\top)^{-1}b$ (the right pseudoinverse formula). Verify numerically.
- Explore the Moore–Penrose pseudoinverse `np.linalg.pinv` for all four cases.

Lab 5. Polynomial Regression

Suggested week: Weeks 5–6 **Notebook:** `lab05_polynomial_regression.ipynb` **Prerequisites:** Notes pages 22–23, §2.1

Background. Given N data points (x_i, y_i), fitting a degree-d polynomial amounts to solving an overdetermined linear system $X\beta = y$ where X is the *Vandermonde matrix* $X_{ij} = x_i^j$. The *normal equations* $X^\top X\beta = X^\top y$ give the least-squares solution. When $d \geq N - 1$ the system is underdetermined and the fit interpolates every point — this is *overfitting*.

Learning objectives.

- Build the Vandermonde feature matrix and understand it as the matrix of a linear map.
- Solve the normal equations and relate them to least-squares approximation.
- Observe overfitting empirically and understand it in terms of rank and kernel.
- Appreciate why the normal equations require an inner product (§2.1).

Tasks.

1. **(Already in notebook)** Generate 100 noisy samples from $y = x^2 + 2x + 1$. Build `feature_matrix(x, d)` and `solve_linear_system(X, y)`. Confirm that degree 2 recovery gives $\beta \approx [1, 2, 1]$.
2. Repeat for degrees $d = 1, 3, 5, 10$. For each, plot the fitted polynomial and report the training error $\|X\beta - y\|$. Describe what happens as d increases past the true degree.
3. Reduce N to 12 data points and fit degree 11. What do you observe? Compute $\mathrm{rank}(X)$ and $\dim\ker(X)$ for this case.
4. Add a *validation set* of 50 new samples from the same polynomial. Plot training error and validation error versus degree $d = 1, \ldots, 15$. Identify the degree at which validation error is minimised.

Extensions.

- Replace the Vandermonde basis with the *Chebyshev* polynomial basis. Does this reduce numerical ill-conditioning? Check via `np.linalg.cond(X)`.
- Implement ridge regression: instead of solving $X^\top X\beta = X^\top y$, solve $(X^\top X + \lambda I)\beta = X^\top y$. How does λ affect the fitted polynomial for large d?

Lab 6. Inner Products, Norms, and Gram–Schmidt

Suggested week: Week 7 **Notebook:** `lab06_gram_schmidt.ipynb` **Prerequisites:** Notes §2.1/§3.1 (pages 24–29)

Background. An *inner product* on V is a bilinear, symmetric, positive-definite map $\langle \cdot, \cdot \rangle : V \times V \to F$. It induces a norm $\|v\| = \sqrt{\langle v, v \rangle}$ and the key inequalities: Cauchy–Schwarz $|\langle u, v \rangle| \leq \|u\| \, \|v\|$ and the triangle inequality. The *Gram–Schmidt* process takes any basis $\{v_1, \ldots, v_n\}$ and produces an orthonormal basis $\{q_1, \ldots, q_n\}$ spanning the same space; this is the QR decomposition $A = QR$.

Learning objectives.

- Verify inner product axioms for standard and non-standard products.
- Implement Gram–Schmidt and understand its geometric meaning.
- Compute the QR decomposition and connect it to least-squares regression.
- Work with the L^2 inner product on P_n using numerical integration.

Tasks.

1. Implement `gram_schmidt(A)` that takes an $n \times k$ matrix (columns = basis vectors) and returns Q (orthonormal columns) and R (upper-triangular). Verify $A = QR$ and $Q^\top Q = I_k$.
2. Apply Gram–Schmidt to the monomial basis $\{1, x, x^2, x^3\}$ on $[-1, 1]$ using the L^2 inner product $\langle f, g \rangle = \int_{-1}^{1} f(x)g(x)\,dx$ (use `scipy.integrate.quad`). The result should approximate the *Legendre polynomials*; verify against `scipy.special.legendre`.
3. Use the QR decomposition to solve the polynomial regression problem from Lab 5. The formula $\beta = R^{-1}Q^\top y$ is numerically more stable than normal equations. Compare condition numbers of $X^\top X$ and R.
4. Verify Cauchy–Schwarz numerically: generate 1000 random pairs $(u, v) \in \mathbb{R}^{10}$ and confirm $|\langle u, v \rangle| \leq \|u\|\|v\|$. Plot the distribution of the ratio $|\langle u, v \rangle|/(\|u\|\|v\|)$.

Extensions.

- Implement the *modified* Gram–Schmidt (MGS) algorithm and compare its numerical stability to classical Gram–Schmidt on a nearly-dependent set of vectors.
- Show that orthogonal projection onto $W = \mathrm{col}(Q)$ equals $QQ^\top$. Apply this to project a random vector onto a 3-dimensional subspace.

Lab 7. The Four Fundamental Subspaces

Suggested week: Weeks 7–8 **Notebook:** `lab07_four_subspaces.ipynb` **Prerequisites:** Notes pages 30–32

Background. Every matrix $A \in \mathbb{R}^{m \times n}$ of rank r determines four subspaces:

$$\text{column space } \mathrm{col}(A) \subseteq \mathbb{R}^m, \quad \text{null space } \ker(A) \subseteq \mathbb{R}^n,$$

$$\text{row space } \operatorname{row}(A) \subseteq \mathbb{R}^n, \quad \text{left null space } \ker(A^\top) \subseteq \mathbb{R}^m.$$

By the Rank–Nullity theorem (and its transpose version): $\dim \operatorname{col}(A) = \dim \operatorname{row}(A) = r$, $\dim \ker(A) = n - r$, $\dim \ker(A^\top) = m - r$. The orthogonal decompositions $\mathbb{R}^n = \operatorname{row}(A) \oplus \ker(A)$ and $\mathbb{R}^m = \operatorname{col}(A) \oplus \ker(A^\top)$ are the core of the *fundamental theorem of linear algebra*.

Learning objectives.

- Compute all four fundamental subspaces for a given matrix.
- Verify the orthogonal complement relationships numerically.
- Visualise the two decompositions geometrically in low dimensions.
- Connect the four subspaces to the solvability of $Ax = b$.

Tasks.

1. Write `four_subspaces(A)` that returns orthonormal bases for $\operatorname{col}(A)$, $\ker(A)$, $\operatorname{row}(A)$, $\ker(A^\top)$ using `scipy.linalg.null_space` and `scipy.linalg.orth`. Verify dimensions satisfy the Rank–Nullity theorem.
2. For a 4×5 matrix A of rank 2, verify the orthogonality relations: (a) every row-space vector is orthogonal to every null-space vector, (b) every column-space vector is orthogonal to every left-null vector.
3. Given $b \in \mathbb{R}^m$, decompose $b = b_{\text{col}} + b_{\ker^\top}$ (projection onto column space + projection onto left null space). Verify that $Ax = b$ is solvable iff $b_{\ker^\top} = 0$.
4. For a 3×3 rank-2 matrix, draw the four subspaces in $\mathbb{R}^3$ using `matplotlib` 3D plots (planes and lines through the origin).

Extensions.

- Show that $A^\top A$ and A have the same null space and the same row space. Use this to prove that the normal equations always have a solution.
- Compute the four subspaces for the Hilbert matrix from Lab 3 using exact (symbolic) arithmetic vs floating-point; compare dimensions.

Lab 8. Change of Basis

Suggested week: Week 8 **Notebook:** `lab08_change_of_basis.ipynb` **Prerequisites:** Notes §4.1 (pages 33–34)

Background. If $\mathcal{B} = \{b_1, \ldots, b_n\}$ is a basis of $\mathbb{R}^n$ and $P = [b_1 \mid \cdots \mid b_n]$ is the *change-of-basis matrix*, then any vector v has coordinates $[v]_{\mathcal{B}} = P^{-1}v$ in $\mathcal{B}$. A linear map T with matrix A in the standard basis has matrix $A' = P^{-1}AP$ in $\mathcal{B}$. Matrices related by $A' = P^{-1}AP$ are called *similar*; they represent the *same* linear map in different bases.

Learning objectives.

- Find the change-of-basis matrix P and compute coordinates in a new basis.
- Compute $A' = P^{-1}AP$ and interpret it as the same map in a different basis.
- Use an eigenbasis to diagonalise a 2×2 matrix by hand and verify in code.
- Apply change of basis to compute matrix powers efficiently via $A^k = P\Lambda^k P^{-1}$.

Tasks.

1. Given basis $\mathcal{B} = \{(1,1),(1,-1)\}/\sqrt{2}$, find the coordinates of $(3,1)$ in $\mathcal{B}$. Write a general function `coords(v, B)` returning $P^{-1}v$.
2. For $A = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$, find eigenvalues and eigenvectors by hand and build P. Compute $A' = P^{-1}AP$ and confirm it is diagonal.
3. Use $A^k = P\Lambda^k P^{-1}$ to compute A^{20} for the matrix above. Compare against `np.linalg.matrix_power(A, 20)`.
4. The Fibonacci sequence satisfies $F_{n+1} = F_n + F_{n-1}$. Write it as $\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = M^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ with $M = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$. Diagonalise M and derive the closed-form Binet formula. Verify for $n = 1, \ldots, 20$.

Extensions.

- For a 3×3 symmetric matrix, diagonalise by Gram–Schmidt on its eigenvectors to get an *orthogonal* P (i.e. $P^{-1} = P^\top$).
- Show that $\mathrm{tr}(A) = \mathrm{tr}(A')$ and $\det(A) = \det(A')$ for any change of basis, connecting to eigenvalue sums and products.

Lab 9. Eigenvalues via Rayleigh Quotients

Suggested week: Week 9 **Notebook:** `lab09_rayleigh_quotients.ipynb` **Prerequisites:** Notes §4.2 (pages 35–38)

Background. For a symmetric matrix A, the *Rayleigh quotient* is $R(x) = x^\top Ax/x^\top x$. Its minimum and maximum over the unit sphere equal the smallest and largest eigenvalues, respectively (Courant–Fischer theorem, a consequence of the Spectral Theorem from §4.3). This variational characterisation motivates a practical algorithm: minimise R by gradient descent on the sphere, then *deflate* by projecting out the found eigenvector and repeating.

Learning objectives.

- Implement gradient descent on the unit sphere to find extreme eigenvalues.
- Implement deflation to recover the full spectrum without determinants.
- Understand why $\det(A - \lambda I) = 0$ is impractical for large matrices.
- Verify results against `numpy.linalg.eigh`.

Tasks.

1. **(Already in notebook)** Construct a 10×10 random symmetric matrix. Run `min_eigen_via_gradient` and compare with `np.linalg.eigh`.
2. **(Already in notebook)** Implement `compute_k_smallest` using deflation and recover the full spectrum; measure the L^2 error against NumPy.
3. **(New)** Study convergence: plot $|R(x_t) - \lambda_{\min}|$ versus gradient-descent step t for learning rates $\eta \in \{0.001, 0.01, 0.1\}$. What is the effect on speed and stability?

4. **(New)** Apply the algorithm to a 100×100 random symmetric matrix. Report wall-clock time versus `np.linalg.eigh`. When would the gradient-descent approach be preferred in practice?

Extensions.

- Implement the *power iteration* method ($x \leftarrow Ax/\|Ax\|$) and compare its convergence to gradient descent on the Rayleigh quotient.
- Show that deflation by projection is equivalent to restricting A to the orthogonal complement of the found eigenvector (invariant subspace property).

Lab 10. SVD: Geometry and Image Compression

Suggested week: Week 10 **Notebook:** `lab10_svd.ipynb` **Prerequisites:** Notes §4.3 (page 44), forward pointer to SVD

Background. Every matrix $A \in \mathbb{R}^{m\times n}$ factors as $A = U\Sigma V^\top$ where $U \in \mathbb{R}^{m\times m}$, $V \in \mathbb{R}^{n\times n}$ are orthogonal and Σ is diagonal with non-negative entries $\sigma_1 \geq \cdots \geq \sigma_r > 0$ (the *singular values*). The *Eckart–Young theorem* says $A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top$ is the best rank-k approximation to A in the Frobenius norm.

Learning objectives.

- Understand the SVD geometrically as mapping the unit sphere to an ellipsoid.
- Apply the Eckart–Young theorem to image compression.
- Analyse the distribution of singular values and their energy content.
- Connect SVD to the spectral decomposition of $A^\top A$ and $AA^\top$.

Tasks.

1. **(Already in notebook)** For a 2×2 matrix, visualise how A maps the unit circle to an ellipse; overlay the singular vectors v_i and images $\sigma_i u_i$.
2. **(Already in notebook)** For a 50×50 random matrix, plot Frobenius reconstruction error $\|M - M_k\|_F$ versus rank k.
3. **(Already in notebook)** Apply SVD compression to grayscale images (astronaut, chelsea, coffee, camera) at ranks 5, 20, 60. Plot singular-value histograms and energy-per-decile charts.
4. **(New)** For each image, find the smallest rank k^* such that $\sum_{i=1}^{k^*} \sigma_i^2 / \sum_i \sigma_i^2 \geq 0.95$. Report k^* and the compression ratio $k^*(m+n)/(mn)$.

Extensions.

- Verify $A^\top A = V\Sigma^2 V^\top$: the eigenvalues of $A^\top A$ are the squared singular values of A.
- For a colour image (3 channels), apply SVD to each channel independently and reconstruct; compare quality to greyscale compression.

Lab 11. Image Processing as Linear Algebra

Suggested week: Weeks 10–11 **Notebook:** `lab11_edge_detection.ipynb` **Prerequisites:** Notes §1.9 (pages 9–10), Lab 10

Background. An image $X \in \mathbb{R}^{m\times n}$ can be viewed as a vector in $\mathbb{R}^{mn}$. *Convolution* with a kernel K is a linear map; its matrix is a (block) Toeplitz matrix. Finite-difference operators (Sobel, Laplacian) are sparse instances. The diffusion PDE $\partial_t X = \alpha\Delta X + \beta(X - \bar{X})$ discretises to $X \leftarrow X + \alpha\Delta X + \beta(X - \bar{X})$, a repeated linear update that smooths and sharpens simultaneously.

Learning objectives.

- Recognise convolution as a linear map and identify its matrix.
- Implement finite-difference gradient operators for edge detection.
- Implement and tune the diffusion-evolution edge detector.
- Compare handcrafted linear operators with library methods (Canny, Felzenszwalb).

Tasks.

1. **(Already in notebook)** Apply a 9×9 averaging kernel to the Mona Lisa image using `scipy.ndimage.convolve`. Write the corresponding Toeplitz matrix for a 1-D slice and verify $(\text{matrix}) \cdot (\text{vector}) = \text{convolve}(\text{1-D signal}, \text{kernel})$.
2. **(Already in notebook)** Compute the gradient magnitude $\|\nabla X\| = \sqrt{(\partial_x X)^2 + (\partial_y X)^2}$ using central finite differences. Threshold to obtain an edge map.
3. **(Already in notebook)** Implement the diffusion-evolution edge detector: iterate $X \leftarrow X + \alpha\Delta X + \beta(X - \bar{X})$, binarise at threshold t, and extract boundaries. Experiment with α, β, t on at least two different images.
4. **(New)** Express the 1-D Laplacian on n points as an $n\times n$ tridiagonal matrix L. Compute its eigenvalues and eigenvectors. Show that the eigenvectors are the discrete cosine modes $v_k(j) = \cos(\pi k j/n)$.

Extensions.

- Implement the *Sobel operator* as a proper 3×3 matrix kernel. Compute its frequency response by taking the DFT.
- For the diffusion step $X \leftarrow (I + \alpha L + \beta(I - \bar{\cdot}))X$, write the update explicitly as a single matrix and find its eigenvalues. What controls stability?

Lab 12. Matrix Denoising via SVD

Suggested week: Week 11 **Notebook:** `lab12_denoising.ipynb` **Prerequisites:** Notes §4.3, Lab 10

Background. In many applications one observes $M = A_{\text{signal}} + E$ where A_{signal} is low-rank (the signal) and E is random noise. SVD truncation $\hat{A}_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top$ recovers the signal when k is chosen correctly. The *Marchenko–Pastur law* from random matrix theory gives a theoretical threshold: if E has i.i.d. $\mathcal{N}(0, \sigma^2)$ entries in $\mathbb{R}^{m\times n}$ with $m \le n$, then all noise singular values lie below $\sigma(\sqrt{m} + \sqrt{n})$ with high probability. An *elbow* in the singular-value plot gives a data-driven alternative.

Learning objectives.

- Construct a rank-r signal plus Gaussian noise and recover the signal by SVD truncation.
- Apply the Marchenko–Pastur threshold to choose the truncation rank automatically.
- Detect the elbow in the singular-value spectrum as an alternative rank estimator.
- Evaluate recovery quality via the Frobenius norm.

Tasks.

1. Construct a rank-3 signal $A = UV^\top$ where $U \in \mathbb{R}^{100\times 3}$, $V \in \mathbb{R}^{200\times 3}$ have random orthonormal columns scaled by singular values $[10, 5, 2]$. Add noise $E \sim \mathcal{N}(0, \sigma^2)$ with $\sigma = 0.5$. Plot the singular values of $M = A + E$ and mark the Marchenko–Pastur threshold $\hat{\sigma}(\sqrt{m} + \sqrt{n})$ (use σ known).
2. Truncate at $k = 1, 2, 3, 4, 5$ and for each compute $\|\hat{A}_k - A\|_F$. Show that $k = 3$ achieves minimum error.
3. Implement `elbow_rank(s)` that finds the elbow of the singular-value curve $\sigma_1 \geq \sigma_2 \geq \cdots$ using the *knee-point* heuristic (maximum perpendicular distance from the line connecting first and last points). Compare its output to the Marchenko–Pastur threshold for several noise levels.
4. Apply SVD denoising to a real image: add synthetic Gaussian noise at levels $\sigma \in \{0.05, 0.1, 0.2\}$ and denoise at the Marchenko–Pastur threshold. Report PSNR (peak signal-to-noise ratio) before and after denoising.

Extensions.

- Implement the *optimal singular value threshold* from Gavish & Donoho (2014): $\tau = (4/\sqrt{3})\sqrt{n}\sigma$. Compare to Marchenko–Pastur on your synthetic data.
- Apply denoising to a real noisy dataset (e.g. collaborative filtering: a partially observed user–item rating matrix). Use the denoised matrix to predict missing entries; evaluate RMSE.